KeenPay

Agentic commerce with bounded AI

KeenPay PostgreSQL schema (canonical)

Database schema (DDL)

01 September 2026

https://github.com/ambitiouss22/KeenPay

KeenPay PostgreSQL schema (canonical)

```sql
-- Postgres 15+. Run: psql $DATABASE_URL -f docs/SCHEMA.sql
--
-- Single source of truth for DDL. Merges core commerce schema, auth tables,
-- and design documentation previously split across SCHEMA.sql, auth migration,
-- and KeenPay_Database_Schema PDF.
--
-- Design principles:
--   - Integer paise (BIGINT/INTEGER) for all money — no floats
--   - AI may propose offers in negotiation_sessions; only policy-approved amounts reach orders
--   - audit_logs is append-only; UPDATE/DELETE blocked by database trigger
--   - Every payment link binds to guardrail_decision_id + offer_version
--   - Webhook events deduplicated by unique event_id
--   - Transaction Passport is derived (no separate table) from sessions, orders, audit_logs
--
-- Table groups:
--   Catalog          -> products
--   Agentic checkout -> negotiation_sessions, langgraph_checkpoints
--   Commerce         -> orders, inventory_holds
--   Control & safety -> escalation_tickets
--   Payments         -> webhook_events
--   Auth             -> users, refresh_tokens, api_keys, auth_audit_log
--   Audit            -> audit_logs (append-only)
--
-- State transitions:
--   negotiation_sessions.status:
--     active -> negotiating -> awaiting_confirmation (guardrail APPROVED)
--     awaiting_confirmation -> payment_pending (user confirmed)
--     payment_pending -> paid (webhook verified)
--     * -> escalated (guardrail ESCALATED) | closed (terminal)
--   orders.status:
--     pending -> paid | expired | cancelled | payment_disputed
-- ================================================================================

BEGIN;

-- Extensions
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
```

SQL

ENUM TYPES

```
CREATE TYPE order_status AS ENUM (
    'pending',
    'paid',
    'expired',
    'cancelled',
    'payment_disputed'
);

CREATE TYPE negotiation_session_status AS ENUM (
    'active',
    'negotiating',
    'awaiting_confirmation',
    'payment_pending',
    'paid',
    'escalated',
    'closed'
);

CREATE TYPE guardrail_outcome AS ENUM (
    'APPROVED',
    'REJECTED',
    'ESCALATED'
);

CREATE TYPE audit_actor AS ENUM (
    'agent',
    'policy_engine',
    'user',
    'system',
    'webhook',
    'human'
);

CREATE TYPE user_role AS ENUM (
    'shopper',
    'support_agent',
    'manager',
    'admin',
    'service'
);

CREATE TYPE auth_event_type AS ENUM (
    'login_success',
    'login_failed',
    'token_issued',
    'token_refreshed',
    'token_revoked',
    'api_key_used',
    'password_changed',
    'account_locked'
);
```

SQL

PRODUCTS

```sql
-- Merchant catalog. Price and cost basis are read by the policy engine for
-- margin guardrails. The AI runtime may read active products but cannot mutate prices.

CREATE TABLE products (
    id                  VARCHAR(64) PRIMARY KEY,
    sku                 VARCHAR(64) NOT NULL,
    merchant_id         VARCHAR(64) NOT NULL DEFAULT 'merchant_keen',
    name                VARCHAR(255) NOT NULL,
    description         TEXT,
    list_price_paise    INTEGER NOT NULL CHECK (list_price_paise >= 0),
    cost_paise          INTEGER NOT NULL CHECK (cost_paise >= 0),
    wholesale_paise     INTEGER CHECK (wholesale_paise IS NULL OR wholesale_paise >= 0),
    quantity_on_hand    INTEGER NOT NULL DEFAULT 0 CHECK (quantity_on_hand >= 0),
    quantity_reserved   INTEGER NOT NULL DEFAULT 0 CHECK (quantity_reserved >= 0),
    attributes          JSONB NOT NULL DEFAULT '{}'::jsonb,
    search_vector       TSVECTOR GENERATED ALWAYS AS (
        to_tsvector(
            'english',
            coalesce(name, '') || ' ' ||
            coalesce(description, '') || ' ' ||
            coalesce(sku, '') || ' ' ||
            coalesce(attributes::text, '')
        )
    ) STORED,
    active              BOOLEAN NOT NULL DEFAULT TRUE,
    created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    CONSTRAINT products_sku_merchant_unique UNIQUE (merchant_id, sku),
    CONSTRAINT products_reserved_lte_on_hand CHECK (quantity_reserved <= quantity_on_hand)
);

COMMENT ON TABLE products IS 'Catalog; cost_paise feeds RULE_MIN_MARGIN';
COMMENT ON COLUMN products.cost_paise IS 'Used by RULE_MIN_MARGIN policy evaluation';
COMMENT ON COLUMN products.attributes IS 'Structured attrs: color, size, category, etc.';

CREATE INDEX idx_products_merchant_active ON products (merchant_id, active);
CREATE INDEX idx_products_sku ON products (sku);
CREATE INDEX idx_products_search ON products USING GIN (search_vector);
CREATE INDEX idx_products_attributes ON products USING GIN (attributes jsonb_path_ops);

-- Computed available quantity (on_hand - reserved) — use in queries:
-- (quantity_on_hand - quantity_reserved) AS quantity_available
```

SQL

NEGOTIATION SESSIONS

```sql
-- Live checkout session. Mirrors LangGraph KeenPayState: intent, offers,
-- guardrail binding, and user confirmation gate. No payment link without
-- APPROVED guardrail + user_confirmed_payment.

CREATE TABLE negotiation_sessions (
    id                      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    merchant_id             VARCHAR(64) NOT NULL DEFAULT 'merchant_keen',
    user_id                 VARCHAR(64),
    status                  negotiation_session_status NOT NULL DEFAULT 'active',
    negotiation_round       INTEGER NOT NULL DEFAULT 0 CHECK (negotiation_round >= 0),
    offer_version           INTEGER NOT NULL DEFAULT 0 CHECK (offer_version >= 0),

    -- Intent & catalog (latest snapshot)
    parsed_intent           JSONB,
    search_results          JSONB NOT NULL DEFAULT '[]'::jsonb,
    selected_line_items     JSONB NOT NULL DEFAULT '[]'::jsonb,

    -- Offers
    proposed_offer          JSONB,
    approved_offer          JSONB,

    -- Guardrail binding
    guardrail_decision      guardrail_outcome,
    guardrail_decision_id   UUID,
    guardrail_detail        JSONB,
    rejection_reasons       JSONB NOT NULL DEFAULT '[]'::jsonb,

    -- User confirmation gate
    user_confirmed_payment  BOOLEAN NOT NULL DEFAULT FALSE,
    user_confirmed_at       TIMESTAMPTZ,

    -- Deterministic totals
    final_amount_paise      INTEGER CHECK (final_amount_paise IS NULL OR final_amount_paise > 0),
    currency                CHAR(3) NOT NULL DEFAULT 'INR',

    -- Security
    anomaly_flags           JSONB NOT NULL DEFAULT '[]'::jsonb,
    security_block          BOOLEAN NOT NULL DEFAULT FALSE,

    -- LangGraph checkpoint reference
    langgraph_thread_id     UUID NOT NULL,

    -- Metadata
    metadata                JSONB NOT NULL DEFAULT '{}'::jsonb,
    created_at              TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at              TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    closed_at               TIMESTAMPTZ,

    CONSTRAINT negotiation_sessions_currency_inr CHECK (currency = 'INR')
);

COMMENT ON TABLE negotiation_sessions IS 'LangGraph session mirror; see ARCHITECTURE.md for state fields';
COMMENT ON COLUMN negotiation_sessions.langgraph_thread_id IS 'LangGraph checkpointer thread_id (equals id by convention)';
COMMENT ON COLUMN negotiation_sessions.guardrail_detail IS 'Full GuardrailDecision JSON including per-rule results';

CREATE INDEX idx_negotiation_sessions_user ON negotiation_sessions (user_id, created_at DESC);
CREATE INDEX idx_negotiation_sessions_status ON negotiation_sessions (status) WHERE status NOT IN ('closed', 'paid');
CREATE INDEX idx_negotiation_sessions_merchant ON negotiation_sessions (merchant_id, created_at DESC);
CREATE INDEX idx_negotiation_sessions_guardrail_decision ON negotiation_sessions (guardrail_decision_id) WHERE guardrail_decision_id IS NOT NULL;
```

SQL

ORDERS

```
-- Frozen purchase created only after guardrail APPROVED + user confirmation.
-- Amounts and line_items are immutable after insert. Binds to Razorpay payment link.

CREATE TABLE orders (
    id                          VARCHAR(64) PRIMARY KEY,
    session_id                  UUID NOT NULL REFERENCES negotiation_sessions(id) ON DELETE RESTRICT,
    merchant_id                 VARCHAR(64) NOT NULL DEFAULT 'merchant_keen',
    user_id                     VARCHAR(64),
    status                      order_status NOT NULL DEFAULT 'pending',

    -- Financial (immutable after creation)
    subtotal_paise              INTEGER NOT NULL CHECK (subtotal_paise >= 0),
    discount_amount_paise       INTEGER NOT NULL DEFAULT 0 CHECK (discount_amount_paise >= 0),
    final_amount_paise          INTEGER NOT NULL CHECK (final_amount_paise > 0),
    currency                    CHAR(3) NOT NULL DEFAULT 'INR',

    -- Line items snapshot at order time
    line_items                  JSONB NOT NULL,

    -- Guardrail traceability
    guardrail_decision_id       UUID NOT NULL,
    offer_version               INTEGER NOT NULL CHECK (offer_version >= 1),
    policy_version              VARCHAR(32) NOT NULL,

    -- Razorpay
    razorpay_payment_link_id    VARCHAR(64),
    razorpay_payment_link_url   TEXT,
    razorpay_payment_id         VARCHAR(64),
    razorpay_order_id           VARCHAR(64),
    payment_link_expires_at     TIMESTAMPTZ,

    -- Idempotency
    idempotency_key             VARCHAR(128) NOT NULL,

    -- Timestamps
    created_at                  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at                  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    paid_at                     TIMESTAMPTZ,
    expired_at                  TIMESTAMPTZ,
    cancelled_at                TIMESTAMPTZ,

    CONSTRAINT orders_currency_inr CHECK (currency = 'INR'),
    CONSTRAINT orders_idempotency_unique UNIQUE (idempotency_key)
);

COMMENT ON TABLE orders IS 'Created only after guardrail APPROVED and user_confirmed_payment';
COMMENT ON COLUMN orders.line_items IS 'Array of {sku, name, quantity, unit_price_paise, line_total_paise}';
COMMENT ON COLUMN orders.guardrail_decision_id IS 'Links order amount to policy evaluation audit trail';

CREATE INDEX idx_orders_session ON orders (session_id);
CREATE INDEX idx_orders_status ON orders (status, created_at DESC);
CREATE INDEX idx_orders_user ON orders (user_id, created_at DESC);
CREATE INDEX idx_orders_razorpay_link ON orders (razorpay_payment_link_id) WHERE razorpay_payment_link_id IS NOT NULL;
CREATE INDEX idx_orders_razorpay_payment ON orders (razorpay_payment_id) WHERE razorpay_payment_id IS NOT NULL;
CREATE UNIQUE INDEX idx_orders_one_pending_per_session ON orders (session_id) WHERE status = 'pending';
```

SQL

AUDIT LOGS (append-only)

```sql
-- Tamper-evident ledger. Every money action and guardrail evaluation writes a row
-- with input_snapshot and output_snapshot. UPDATE/DELETE blocked by trigger.

CREATE TABLE audit_logs (
    id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id          UUID REFERENCES negotiation_sessions(id) ON DELETE SET NULL,
    order_id            VARCHAR(64) REFERENCES orders(id) ON DELETE SET NULL,
    merchant_id         VARCHAR(64) NOT NULL DEFAULT 'merchant_keen',

    actor               audit_actor NOT NULL,
    action              VARCHAR(128) NOT NULL,

    -- Correlation
    decision_id         UUID,
    offer_version       INTEGER,
    idempotency_key     VARCHAR(128),
    trace_event_id      UUID,

    -- Snapshots for replay / compliance
    input_snapshot      JSONB NOT NULL DEFAULT '{}'::jsonb,
    output_snapshot     JSONB NOT NULL DEFAULT '{}'::jsonb,
    trace_metadata      JSONB NOT NULL DEFAULT '{}'::jsonb,

    -- Request context
    ip_address          INET,
    user_agent          TEXT,

    created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

COMMENT ON TABLE audit_logs IS 'Append-only; money actions and guardrail evals. Do not UPDATE.';
COMMENT ON COLUMN audit_logs.trace_metadata IS 'LangGraph node, duration_ms, rule_id, anomaly_score, etc.';
COMMENT ON COLUMN audit_logs.input_snapshot IS 'e.g. proposed_offer, policy_version, user_message hash';
COMMENT ON COLUMN audit_logs.output_snapshot IS 'e.g. guardrail outcome, payment_link_id, error codes';

CREATE INDEX idx_audit_logs_session_created ON audit_logs (session_id, created_at DESC);
CREATE INDEX idx_audit_logs_order ON audit_logs (order_id, created_at DESC) WHERE order_id IS NOT NULL;
CREATE INDEX idx_audit_logs_action ON audit_logs (action, created_at DESC);
CREATE INDEX idx_audit_logs_decision ON audit_logs (decision_id) WHERE decision_id IS NOT NULL;
CREATE INDEX idx_audit_logs_actor_action ON audit_logs (actor, action, created_at DESC);
CREATE INDEX idx_audit_logs_trace_metadata ON audit_logs USING GIN (trace_metadata jsonb_path_ops);

-- Prevent updates/deletes via trigger (append-only enforcement)
CREATE OR REPLACE FUNCTION audit_logs_deny_mutation()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION 'audit_logs is append-only';
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_audit_logs_no_update
    BEFORE UPDATE ON audit_logs
    FOR EACH ROW EXECUTE FUNCTION audit_logs_deny_mutation();

CREATE TRIGGER trg_audit_logs_no_delete
    BEFORE DELETE ON audit_logs
    FOR EACH ROW EXECUTE FUNCTION audit_logs_deny_mutation();
```

SQL

SUPPORTING TABLES

```sql
-- Inventory holds (durable complement to Redis)
CREATE TABLE inventory_holds (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id      UUID NOT NULL REFERENCES negotiation_sessions(id) ON DELETE CASCADE,
    product_id      VARCHAR(64) NOT NULL REFERENCES products(id) ON DELETE RESTRICT,
    sku             VARCHAR(64) NOT NULL,
    quantity        INTEGER NOT NULL CHECK (quantity > 0),
    expires_at      TIMESTAMPTZ NOT NULL,
    released_at     TIMESTAMPTZ,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    CONSTRAINT inventory_holds_active_unique UNIQUE (session_id, sku)
);

CREATE INDEX idx_inventory_holds_expires ON inventory_holds (expires_at) WHERE released_at IS NULL;
CREATE INDEX idx_inventory_holds_product ON inventory_holds (product_id) WHERE released_at IS NULL;

-- Razorpay webhook idempotency
CREATE TABLE webhook_events (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    event_id        VARCHAR(128) NOT NULL,
    event_type      VARCHAR(64) NOT NULL,
    payload         JSONB NOT NULL,
    signature_valid BOOLEAN NOT NULL,
    processed       BOOLEAN NOT NULL DEFAULT FALSE,
    process_result  JSONB,
    order_id        VARCHAR(64) REFERENCES orders(id) ON DELETE SET NULL,
    received_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    processed_at    TIMESTAMPTZ,

    CONSTRAINT webhook_events_event_id_unique UNIQUE (event_id)
);

CREATE INDEX idx_webhook_events_type ON webhook_events (event_type, received_at DESC);
CREATE INDEX idx_webhook_events_unprocessed ON webhook_events (received_at) WHERE processed = FALSE;

-- Human-in-the-loop escalations
CREATE TABLE escalation_tickets (
    id                      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id              UUID NOT NULL REFERENCES negotiation_sessions(id) ON DELETE CASCADE,
    priority                VARCHAR(4) NOT NULL CHECK (priority IN ('P0', 'P1', 'P2')),
    reason_code             VARCHAR(64) NOT NULL,
    status                  VARCHAR(16) NOT NULL DEFAULT 'open'
                            CHECK (status IN ('open', 'assigned', 'resolved', 'expired')),
    assigned_to             VARCHAR(64),
    proposed_offer_snapshot JSONB NOT NULL,
    policy_snapshot         JSONB NOT NULL,
    resolution              VARCHAR(32) CHECK (resolution IN ('approve_override', 'deny', 'counter_offer')),
    override_discount_pct   NUMERIC(5, 2),
    resolver_note           TEXT,
    created_at              TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    resolved_at             TIMESTAMPTZ
);

CREATE INDEX idx_escalation_tickets_status ON escalation_tickets (status, priority, created_at);

-- LangGraph checkpoints (if not using LangGraph's auto-migration)
CREATE TABLE langgraph_checkpoints (
    thread_id       UUID NOT NULL,
    checkpoint_ns   VARCHAR(256) NOT NULL DEFAULT '',
    checkpoint_id   UUID NOT NULL,
    parent_id       UUID,
    checkpoint      JSONB NOT NULL,
    metadata        JSONB NOT NULL DEFAULT '{}'::jsonb,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    PRIMARY KEY (thread_id, checkpoint_ns, checkpoint_id)
);

CREATE INDEX idx_langgraph_checkpoints_thread ON langgraph_checkpoints (thread_id, created_at DESC);
```

SQL

AUTH TABLES

```sql
CREATE TABLE users (
    id                  VARCHAR(64) PRIMARY KEY DEFAULT ('user_' || replace(gen_random_uuid()::text, '-', '')),
    email               VARCHAR(255) NOT NULL,
    password_hash       VARCHAR(255),
    merchant_id         VARCHAR(64) NOT NULL DEFAULT 'merchant_keen',
    role                user_role NOT NULL DEFAULT 'shopper',
    display_name        VARCHAR(255),
    active              BOOLEAN NOT NULL DEFAULT TRUE,
    locked_until        TIMESTAMPTZ,
    failed_login_count  INTEGER NOT NULL DEFAULT 0 CHECK (failed_login_count >= 0),
    last_login_at       TIMESTAMPTZ,
    created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    CONSTRAINT users_email_merchant_unique UNIQUE (merchant_id, email)
);

CREATE INDEX idx_users_merchant_role ON users (merchant_id, role) WHERE active = TRUE;

CREATE TABLE refresh_tokens (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id         VARCHAR(64) NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    token_hash      VARCHAR(64) NOT NULL UNIQUE,
    family_id       UUID NOT NULL DEFAULT gen_random_uuid(),
    expires_at      TIMESTAMPTZ NOT NULL,
    revoked_at      TIMESTAMPTZ,
    replaced_by     UUID REFERENCES refresh_tokens(id),
    user_agent      VARCHAR(512),
    ip_address      INET,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_refresh_tokens_user ON refresh_tokens (user_id) WHERE revoked_at IS NULL;
CREATE INDEX idx_refresh_tokens_expires ON refresh_tokens (expires_at) WHERE revoked_at IS NULL;

CREATE TABLE api_keys (
    id              VARCHAR(64) PRIMARY KEY DEFAULT ('key_' || replace(gen_random_uuid()::text, '-', '')),
    name            VARCHAR(255) NOT NULL,
    key_prefix      VARCHAR(16) NOT NULL,
    key_hash        VARCHAR(64) NOT NULL UNIQUE,
    merchant_id     VARCHAR(64) NOT NULL DEFAULT 'merchant_keen',
    role            user_role NOT NULL DEFAULT 'service',
    scopes          TEXT[] NOT NULL DEFAULT '{}',
    active          BOOLEAN NOT NULL DEFAULT TRUE,
    expires_at      TIMESTAMPTZ,
    last_used_at    TIMESTAMPTZ,
    created_by      VARCHAR(64) REFERENCES users(id),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    revoked_at      TIMESTAMPTZ
);

CREATE INDEX idx_api_keys_prefix ON api_keys (key_prefix) WHERE active = TRUE AND revoked_at IS NULL;

CREATE TABLE auth_audit_log (
    id              BIGSERIAL PRIMARY KEY,
    event_type      auth_event_type NOT NULL,
    user_id         VARCHAR(64) REFERENCES users(id),
    api_key_id      VARCHAR(64) REFERENCES api_keys(id),
    merchant_id     VARCHAR(64),
    ip_address      INET,
    user_agent      VARCHAR(512),
    metadata        JSONB NOT NULL DEFAULT '{}'::jsonb,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_auth_audit_user ON auth_audit_log (user_id, created_at DESC);
CREATE INDEX idx_auth_audit_merchant ON auth_audit_log (merchant_id, created_at DESC);

CREATE OR REPLACE FUNCTION deny_auth_audit_mutation()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION 'auth_audit_log is append-only';
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_auth_audit_no_update
    BEFORE UPDATE OR DELETE ON auth_audit_log
    FOR EACH ROW EXECUTE FUNCTION deny_auth_audit_mutation();
```

SQL

UPDATED_AT TRIGGER

```
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_products_updated_at
    BEFORE UPDATE ON products
    FOR EACH ROW EXECUTE FUNCTION set_updated_at();

CREATE TRIGGER trg_negotiation_sessions_updated_at
    BEFORE UPDATE ON negotiation_sessions
    FOR EACH ROW EXECUTE FUNCTION set_updated_at();

CREATE TRIGGER trg_orders_updated_at
    BEFORE UPDATE ON orders
    FOR EACH ROW EXECUTE FUNCTION set_updated_at();

CREATE TRIGGER trg_users_updated_at
    BEFORE UPDATE ON users
    FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

SQL

SEED DATA (sample catalog + dev users)

```
INSERT INTO products (id, sku, name, description, list_price_paise, cost_paise, quantity_on_hand, attributes) VALUES
    ('prod_001', 'HOODIE-NAVY-M', 'Keen Hoodie Navy M', 'Premium cotton hoodie, navy, medium', 249900, 120000, 50, '{"color": "navy", "size": "M", "category": "apparel"}'),
    ('prod_002', 'HOODIE-NAVY-L', 'Keen Hoodie Navy L', 'Premium cotton hoodie, navy, large', 249900, 120000, 35, '{"color": "navy", "size": "L", "category": "apparel"}'),
    ('prod_003', 'TEE-BLACK-M', 'Keen Tee Black M', 'Organic cotton t-shirt, black, medium', 99900, 45000, 100, '{"color": "black", "size": "M", "category": "apparel"}'),
    ('prod_004', 'CAP-WHITE-OS', 'Keen Cap White', 'Adjustable dad cap, white', 79900, 35000, 80, '{"color": "white", "size": "OS", "category": "accessories"}'),
    ('prod_005', 'BAG-TOTE-NAT', 'Keen Tote Natural', 'Canvas tote bag, natural', 129900, 55000, 40, '{"color": "natural", "category": "accessories"}');

-- Dev users (password: KeenPayDev1! — change in production)
INSERT INTO users (id, email, password_hash, merchant_id, role, display_name)
VALUES
    ('user_dev_shopper', 'shopper@keenpay.dev', '$2b$12$LQv3c1yqBWVHxkd0LHAkCOYz6TtxMQJqhN8/X4.G2oQKHyL4GqK0i', 'merchant_keen', 'shopper', 'Dev Shopper'),
    ('user_dev_support', 'support@keenpay.dev', '$2b$12$LQv3c1yqBWVHxkd0LHAkCOYz6TtxMQJqhN8/X4.G2oQKHyL4GqK0i', 'merchant_keen', 'support_agent', 'Dev Support'),
    ('user_dev_manager', 'manager@keenpay.dev', '$2b$12$LQv3c1yqBWVHxkd0LHAkCOYz6TtxMQJqhN8/X4.G2oQKHyL4GqK0i', 'merchant_keen', 'manager', 'Dev Manager'),
    ('user_dev_admin', 'admin@keenpay.dev', '$2b$12$LQv3c1yqBWVHxkd0LHAkCOYz6TtxMQJqhN8/X4.G2oQKHyL4GqK0i', 'merchant_keen', 'admin', 'Dev Admin')
ON CONFLICT (merchant_id, email) DO NOTHING;

COMMIT;
```

SQL

EXAMPLE QUERIES

```
-- Available inventory for guardrail check:
-- SELECT sku,
--        quantity_on_hand - quantity_reserved AS quantity_available,
--        cost_paise
-- FROM products
-- WHERE sku = 'HOODIE-NAVY-M' AND active = TRUE;

-- Audit replay for a session:
-- SELECT created_at, actor, action, decision_id, output_snapshot
-- FROM audit_logs
-- WHERE session_id = '7c9e6679-7425-40de-944b-e07fc1f90ae7'
-- ORDER BY created_at ASC;

-- Orders awaiting payment:
-- SELECT id, final_amount_paise, razorpay_payment_link_url, payment_link_expires_at
-- FROM orders
-- WHERE status = 'pending' AND payment_link_expires_at > NOW();
```

SQL

PRODUCTION CHECKLIST

```
-- [ ] All money stored as integer paise — no floating point
-- [ ] Payment link requires guardrail_decision=APPROVED + user_confirmed_payment
-- [ ] Idempotency key on every order and cached Razorpay link per offer version
-- [ ] Webhook HMAC verified; duplicate event_id returns 200 without side effect
-- [ ] Webhook amount must equal orders.final_amount_paise or mark payment_disputed
-- [ ] audit_logs append-only (trigger blocks UPDATE/DELETE)
-- [ ] Inventory: quantity_reserved <= quantity_on_hand constraint
-- [ ] LLM never writes to orders or webhook_events — gated Python nodes only
-- [ ] Negotiation capped at 5 rounds before ESCALATED -> escalation_tickets
-- [ ] Secrets in environment / secrets manager — never in database or LLM context
```

SQL

v1.1 ROADMAP (not in current DDL)

```
-- Deferred to keep v1 deployable: separate tenants/users with RLS, standalone
-- policies/authorizations tables, refunds, idempotency_keys cache table,
-- transactional outbox, and GROW campaign tables. v1 encodes policy in
-- MerchantPolicy config and guardrail snapshots in audit_logs JSONB.
```

SQL